UMassAmherst | Manning College of Information & Computer Sciences

Programming Methodology

Lab 10: Asynchronous Programming and Git

Wednesday, April 15th, 2026

With contributions from Atharva Kale, Kevin Chen, Takuto Ban
Git slides adapted from Jaime Davila

Agenda

Weekly Reminders

Motivate Async Programming

Async Exercise
Scavenger Hunt

Use Gradescope

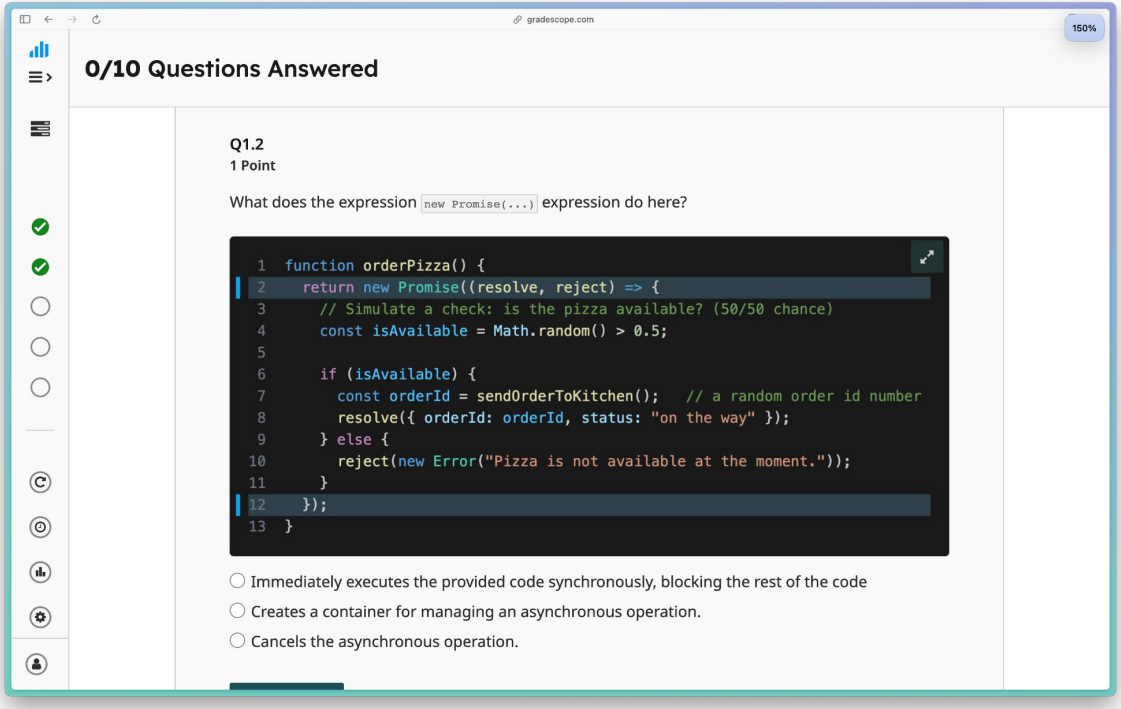
Git Tutorial

Reminders

- Midterm 2 is being graded
- Homework 8 will be released tomorrow (4/16)
  - Due two weeks from today (4/30)
  - Reach out to your teammates ASAP
    - Submit screenshot of email or group chat to Gradescope by 11:59 pm
  - Come to office hours for help!
- If you need to miss lab and have a valid reason according to the syllabus (medical, other personal) please fill out the questionnaire on Canvas before the start time of your lab.
  - Waking up late, bus was late are NOT valid reasons to miss lab.
- Download and submit the lab code now, then again once you're done.

New Lab Format for Today

We have some questions on Gradescope today :)

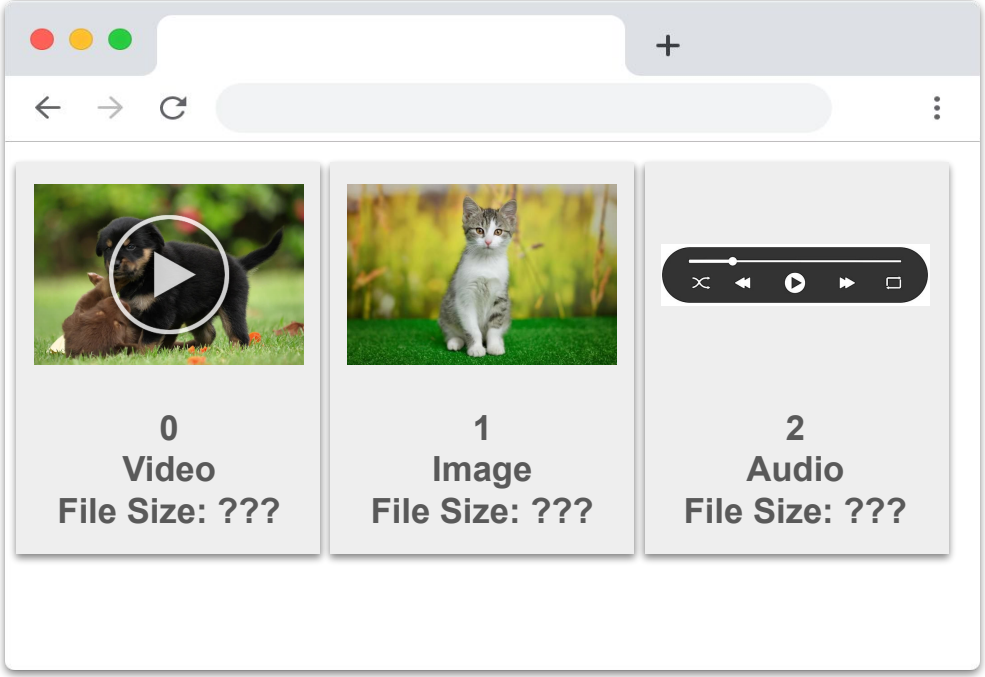


Motivation - Asynchronous Programming

Let's say you are developing a browser responsible for displaying three items:

1) A video
2) An image
3) Some audio

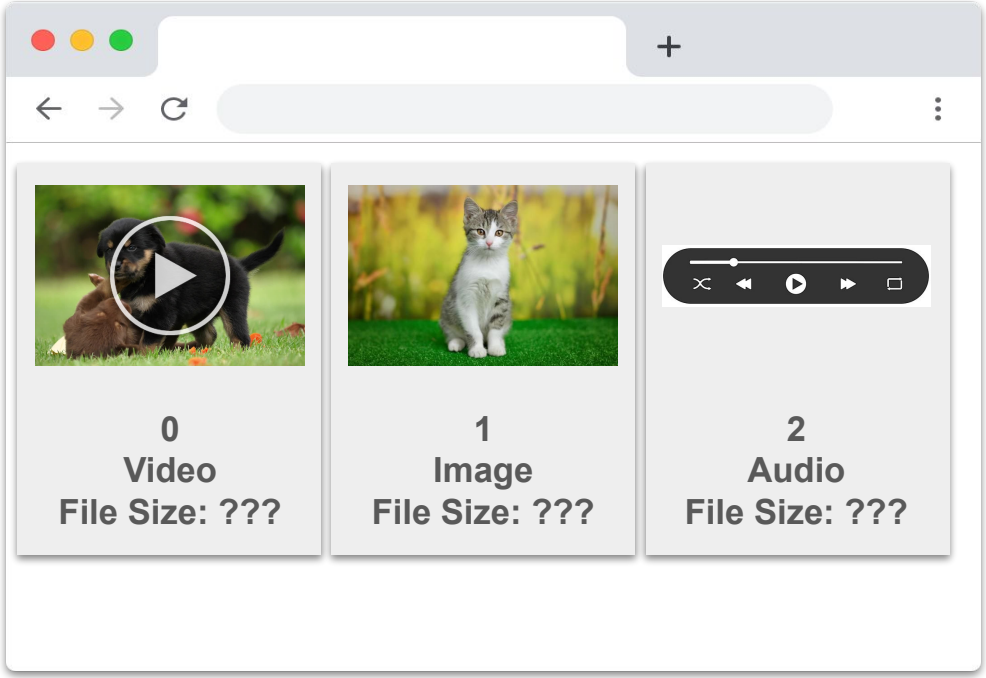
Goal: Give user best user experience



Motivation - Asynchronous Programming

We have three URLs:

```
const urls = [
  "http://catsanddogs.com/video.mp4",
  "http://catsanddogs.com/image.png",
  "http://catsanddogs.com/audio.mp3",
]
```



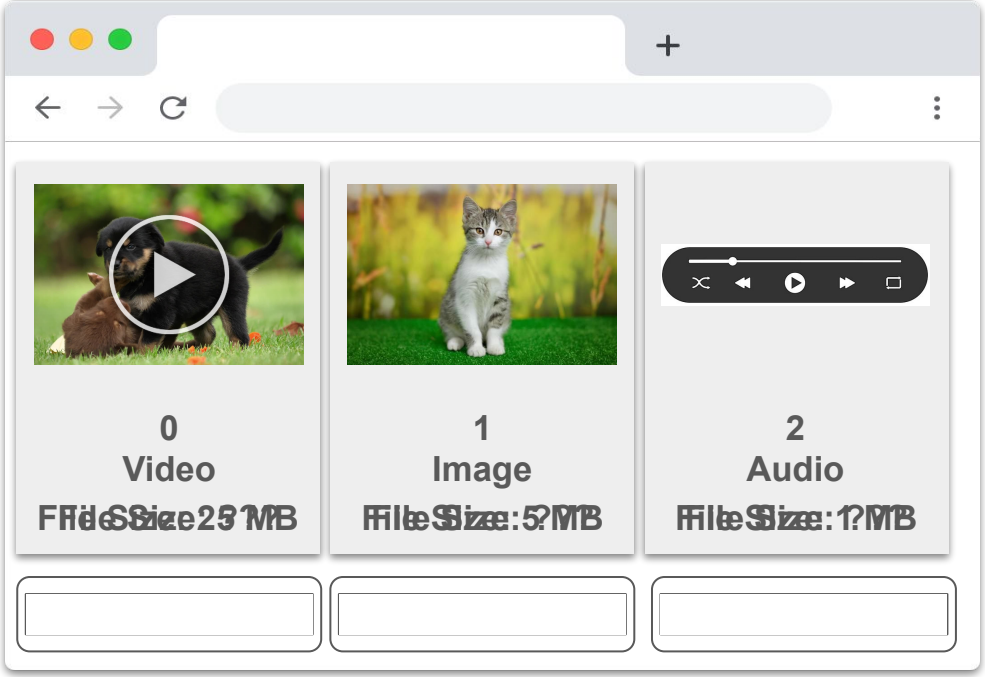
Motivation - Asynchronous Programming

With a basic implementation, we could:

1) Download all assets sequentially and then display them all at once to the user

or....

1) Download an asset and display it sequentially



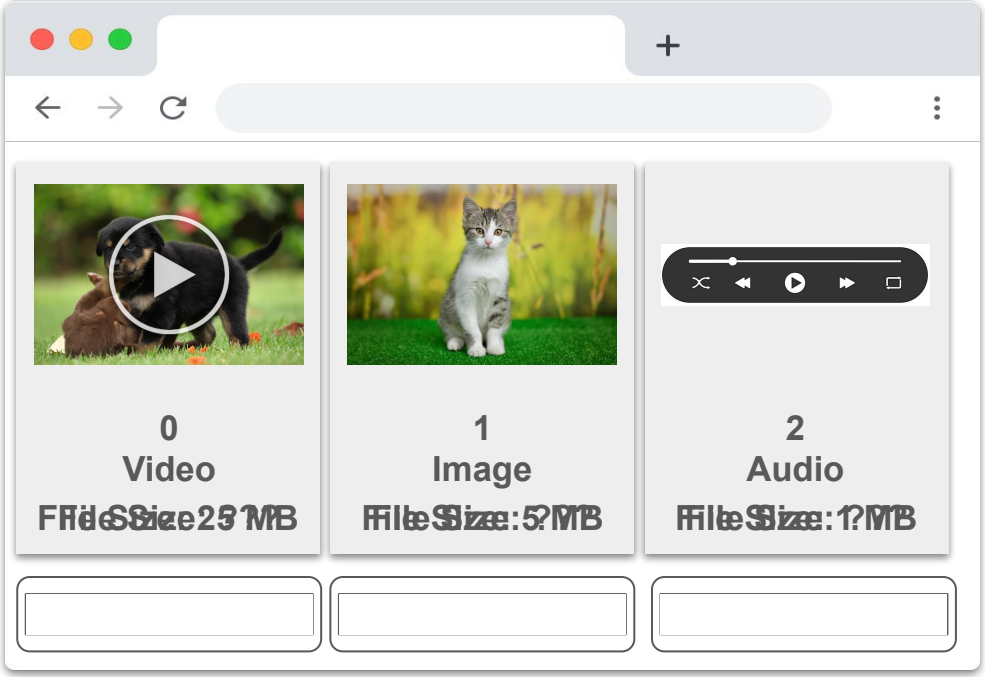
Motivation - Asynchronous Programming

But what if we could start downloading them all at once

AND

Display each right after they finish downloading?

The Promise API lets us do that!



Review: Asynchronous Programming

So far, we have mainly dealt with **synchronous** programming in this class. In general, this means our code runs line by line, and each line will **only** run once the previous line has finished running.

However, in JavaScript/TypeScript (and many other languages), we can also have asynchronous code, which we don't need to wait for before we execute other code.

We can see how asynchronous programming fits in with the example from the previous slides, where we want to download several different files for a website. Instead of waiting for each download to complete, we can begin downloading multiple files all at once!

Review: Promises

The **Promise** class is very useful for asynchronous programming in JS/TS. A Promise object typically represents the results of some asynchronous task.

At any point, a Promise object will be in one of three states:

- **Pending**: no value or result is available yet from the Promise
- **Fulfilled**: the task has completed and the Promise has resolved to a result
- **Rejected**: the task failed and the Promise rejects (usually with an error)

Note: once the Promise objects fulfills or rejects, the object is still a Promise. It does not turn directly into the result of the asynchronous task or the resulting error.

Review: Promises

An example of an asynchronous function in JS/TS is `setTimeout`. If we start a stopwatch when we run the following code, the code will print "`two`" after 1 second, "`three`" after 2 seconds, and then "`one`" after 4 seconds.

```
setTimeout(() => console.log("one"), 4000);
setTimeout(() => console.log("two"), 1000);
setTimeout(() => console.log("three"), 2000);
```

Outputs →

```
'two'
'three'
'one'
```

Now, suppose we have an asynchronous function `timerPromise(n)`, which returns a Promise that resolves to "`hi!`" after n seconds. Then, in the first 5 seconds, `myPromise` is **pending**. After those 5 seconds, `myPromise` **fulfills**.

```
const myPromise = timerPromise(5); // after fulfilling, myPromise is
                                   still a Promise, just resolved to "hi!"
```

Creating Promises

```js
// main.js

const p1 = new Promise((resolve) => {
  // <an operation that takes 10 secs to complete>
  console.log("p1 completed");
});

const p2 = new Promise((resolve) => {
  // <an operation that takes 5 secs to complete>
  console.log("p2 completed");
});

console.log("main.js can continue work here");

```

```
// Console

main.js can continue work here
p2 completed
p1 completed
```

Resolving Promises

```
// Promises can "resolve" to a value

const p1 = new Promise((resolve) => {
  console.log("p1 completed");
  resolve("p1 resolved");
});

```

What can we do with the fulfillment value?

Resolving Promises

```
// We can chain promises using `.then`

const p1 = new Promise((resolve) => {
  console.log("p1 completed");
  // NOTE: resolve is a function that
  // accepts the fulfilled value
  resolve("p1 resolved");
})

const p2 = p1.then(
  // NOTE: fulfilledValue is the SAME
  // as the value we resolved with
  (fulfilledValue) => {
    console.log(fulfilledValue);
  },
);
// NOTE: p2 is ALSO a promise
```

```
// Console

p1 completed
p1 resolved
```

Read the MDN docs for the full specification of `Promise.prototype.then`

Promises that “reject”

```
// Promises can also "reject" with an error

const p1 = new Promise((resolve, reject) => {
  console.log("p1 completed");
  reject("p1 rejected");
});

const p2 = p1.then(
  (fulfilledValue) => {
    // NEVER runs
    console.log("fulfilledValue:", fulfilledValue);
  }
).catch((rejectedValue) => {
  // THIS WILL RUN
  console.log("rejectedValue:", rejectedValue);
});
```

```
// Console

p1 completed
rejectedValue: p1 rejected
```

Read the MDN docs for the full specification of `Promise.prototype.catch`

Review: Promise API

In summary, we can access the result of a Promise using **.then()** or **.catch()**. Suppose we have a variable `myPromise` storing a Promise.

- If `myPromise` **rejects**, then `myPromise.catch(myFunc)` will call the function `myFunc` on the value that `myPromise` rejected with.
- If `myPromise` **fulfills**, then `myPromise.then(myFunc)` will call the function `myFunc` on the value that `myPromise` fulfilled with.

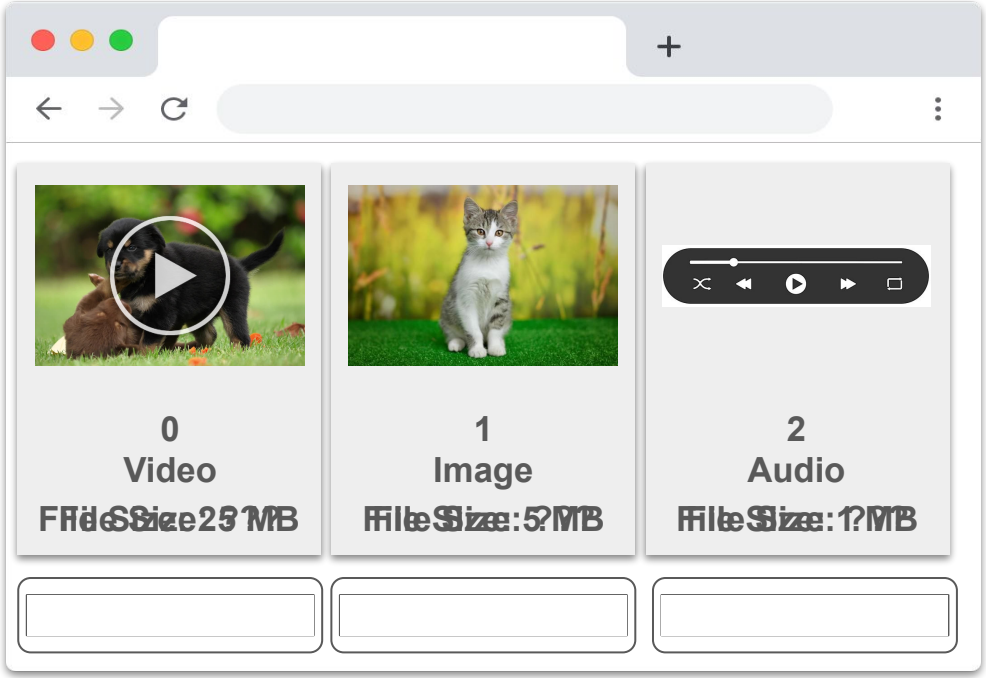
.then() can accept 2 functions, where `myPromise.then(myFunc1, myFunc2)` will call `myFunc1` if `myPromise` **fulfills**, and `myFunc2` if `myPromise` **rejects**.

Also, `.then()` and `.catch()` return a Promise as well, allowing you to chain these methods together.

Basic Promise API

We know how to:

- Create Promises
- Chain Promises
- Handle rejected Promises
- But how do we **start several asynchronous tasks** at once and then **combine the result**?



Exercise: More Promises

Scavenger Hunt!

- **Get the starter code** from the website
- Open exercise in ex.js
- In your groups, **find the password**
- Answer the questions on Gradescope

TAs will walk around to help ensure you are on the right track

Combining Promises

Task: Write a function `countSucc` that **accepts an array of promises** and returns a Promise that fulfills with the number of successful promises from the input array

```
// countSucc: (Promise<T>[]) => Promise<number>
function countSucc(promiseArr) { ... }
```

Definition from the docs:
`Promise.allSettled(promiseArr)`: returns a Promise that always fulfills with an array of objects describing the outcome of each input promise:
`{ status: "fulfilled" | "rejected", value?: someType, reason?: errType }`

Discuss: Using `Promise.allSettled`, how would you implement `countSucc`? Briefly discuss with your group for 3-5 minutes. No need to submit any code!

Combining Promises

Task: Write a function `countSucc` that **accepts an array of promises** and returns a Promise that fulfills with the number of successful promises from the input array

```
// countSucc: (Promise<T>[]) => Promise<number>
function countSucc(promiseArr) {
  return Promise.allSettled(promiseArr)
    .then(results => results.reduce((acc, e) => {
      if (e.status == 'fulfilled') return acc + 1
      else return acc
    }, 0))
}
```

Definition from the docs:
`Promise.allSettled(promiseArr)`:
returns a Promise that always fulfills with an array of objects describing the outcome of each input promise:
`{ status: "fulfilled" | "rejected", value?: someType, reason?: errType }`

Exercise: More Promises

Scavenger Hunt!

- You are tasked with **finding a password** hidden across campus. Time is limited!
- Use Promise API to search for password pieces in a number of locations asynchronously.
- Assemble the password to *save the day!*

Exercise: More Promises

UMassAmherst | Manning College of Information & Computer Sciences

Scavenger Hunt!

You are given the function `searchClueAtLocation`.

Calling this function on certain locations on campus will produce a Promise. This Promise fulfills to a password part and its index in the entire password.

You will be given an array of locations to search for password parts at.

```
const promise = searchClueAtLocation("LGRT")
// returns Promise<{ part: "geese", index: 3 }>
```

Solution: Scavenger Hunt

```
const passwordParts = Array(locations.length).fill("");
const searchPromises = locations.map(searchClueAtLocation);
// ...
```

We begin by initializing an array to store password parts in, such that we can store the password part from each location. We then use the given **searchClueAtLocation** function, calling it on each location using **map**. We store the Promises returned in the array **searchPromises**.

Solution: Scavenger Hunt

```
const passwordParts = Array(locations.length).fill("");
const searchPromises = locations.map(searchClueAtLocation);
Promise.allSettled(searchPromises).then(results => {
    // ...
});
```

Next, we use **Promise.allSettled** to combine all the Promises we obtained into one Promise. This combined Promise will resolve only once all Promises in **searchPromises** are no longer pending.

The combined Promise is also guaranteed to fulfill with an array of objects:
`{ status: "fulfilled" | "rejected", value?: someType, reason?: errType }`

These objects describe the results of each Promise from the array **searchPromises**. Using **.then()**, the array of objects from **Promise.allSettled** is passed in as the argument **results**.

Solution: Scavenger Hunt

```
const passwordParts = Array(locations.length).fill("");
const searchPromises = locations.map(searchClueAtLocation);
Promise.allSettled(searchPromises).then(results => {
    results.forEach(result => {
        if (result.status === "fulfilled") {
            // ...
        }
    });
    // ...
});
```

For each result object in **results**, we only want to examine the ones that correspond to a Promise that fulfilled. We therefore check this by looking at the **status** property of each result object.

Solution: Scavenger Hunt

```
const passwordParts = Array(locations.length).fill("");
const searchPromises = locations.map(searchClueAtLocation);
Promise.allSettled(searchPromises).then(results => {
    results.forEach(result => {
        if (result.status === "fulfilled") {
            const { part, index } = result.value;
            passwordParts[index] = part;
        }
    });
    // ...
});
```

If the result is for a fulfilled Promise, then we know that **result.value** will contain a password part and its index. We therefore extract these from **result.value** and store the password part in our **passwordParts** array.

Solution: Scavenger Hunt

```
const passwordParts = Array(locations.length).fill("")
const searchPromises = locations.map(searchClueAtLocation);
Promise.allSettled(searchPromises).then(results => {
    results.forEach(result => {
        if (result.status === "fulfilled") {
            const { part, index } = result.value
            passwordParts[index] = part
        }
    });
    const password = passwordParts.join("");
    console.log(`Password: ${password}`);
    completeHunt(password);
});
```

Lastly, after going through all the results we obtained from the different locations, we combine the password parts and complete our scavenger hunt!

Lab Feedback

Q3 Anonymous Feedback
0 Points

Since this is a relatively new lab, we would like to hear your anonymous feedback at:
https://forms.gle/QJKSu4kvn8Kt64g68

Please give us feedback! It's very short!

Save Answer

Anonymous Feedback

Version Control With Git

- Step 1: Install Git
  - https://git-scm.com/book/en/v2/Getting-Started-Installing-Git
- Step 2: Learn to Use Git
  - Many tutorials online
    - https://www.w3schools.com/git/
    - https://git-scm.com/docs/gittutorial
    - Hopefully what we see today together will also help.

Version Control With Git

Let git, on your machine, know who you are

```
git config --global user.email "you@example.com"
git config --global user.name "Your Name"
```

Version Control With Git

- Git, local version control: (using the terminal)
  - Make a new directory/folder
  - Change to that folder
  - **`git init`**
  - To check the status of your repository, **`git status`**

```
cics002:~ jaimedavila$ mkdir Git_example
cics002:~ jaimedavila$ cd Git_example/
cics002:Git_example jaimedavila$ git init
Initialized empty Git repository in /Users/jaimedavila/Git_example/.git/
cics002:Git_example jaimedavila$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
cics002:Git_example jaimedavila$
```

Version Control With Git

- Go ahead and make a file
  - And then **`git status`** again

```
cics002:Git_example jaimedavila$ vi sample_file.txt
cics002:Git_example jaimedavila$ cat sample_file.txt
this is juust a sample file
cics002:Git_example jaimedavila$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        sample_file.txt

nothing added to commit but untracked files present (use "git add" to track)
cics002:Git_example jaimedavila$
```

Version Control With Git

- To keep track of that new file, **`git add name_of_file`**
  - And then **`git status`** again

```
cics002:Git_example jaimedavila$ git add sample_file.txt
cics002:Git_example jaimedavila$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   sample_file.txt

cics002:Git_example jaimedavila$
```

Version Control With Git

- Once you are “satisfied for now” with a file, you commit it.
  - **`git commit -am`**
  - You MUST include a description of the commit.
- **`git log`** will show you information regarding past commits.

```
cics002:Git_example jaimedavila$ git commit -am "Adding the first file to the repository"
[main (root-commit) 17f3860] Adding the first file to the repository
 1 file changed, 1 insertion(+)
 create mode 100644 sample_file.txt
cics002:Git_example jaimedavila$ git status
On branch main
nothing to commit, working tree clean
cics002:Git_example jaimedavila$
```

Version Control With Git

- Let's now make a second version of our file(s)
  - This is called branching
  - **`git branch name_of_branch`**
- And switch to that new branch

```
cics002:Git_example jaimedavila$ git branch v2
cics002:Git_example jaimedavila$ git switch v2
Switched to branch 'v2'
cics002:Git_example jaimedavila$ git status
On branch v2
nothing to commit, working tree clean
cics002:Git_example jaimedavila$
```

Version Control With Git

- Now edit that one file you have in the directory
- **`git status`** tells you the file has been modified
  - And that it needs to be added
  - And then committed

```
cics002:Git_example jaimedavila$ vi sample_file.txt
cics002:Git_example jaimedavila$ cat sample_file.txt
this is juust a sample file
this line was added in version 2
cics002:Git_example jaimedavila$ git status
On branch v2
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   sample_file.txt

no changes added to commit (use "git add" and/or "git commit -a")
cics002:Git_example jaimedavila$
```

Version Control With Git

- After committing that change, go back to branch main
  - **`git switch main`**
- Take a look at the file you just modified
  - The change you make on the branch is not there! Where did it go?
- The changes are stored in objects. How, exactly is not important

```
cics002:Git_example jaimedavila$ git add sample_file.txt
cics002:Git_example jaimedavila$ git commit -am "a small version in v2"
[v2 a8bcf88] a small version in v2
 1 file changed, 1 insertion(+)
cics002:Git_example jaimedavila$ git status
On branch v2
nothing to commit, working tree clean
cics002:Git_example jaimedavila$ git switch main
Switched to branch 'main'
cics002:Git_example jaimedavila$ cat sample_file.txt
this is juust a sample file
cics002:Git_example jaimedavila$
```

Version Control With Git

If you go back to the second branch, you will again see the edits you did there.

```
cics002:Git_example jaimedavila$ git status
On branch main
nothing to commit, working tree clean
cics002:Git_example jaimedavila$ cat sample_file.txt
this is juust a sample file
cics002:Git_example jaimedavila$ git switch v2
Switched to branch 'v2'
cics002:Git_example jaimedavila$ cat sample_file.txt
this is juust a sample file
this line was added in version 2
cics002:Git_example jaimedavila$
```

Version Control With Git

- In the main branch, do one more edit to your file.
- It would be great if we could see the differences between those two versions
  - `git diff main..v2`

Version Control With Git

git diff main..v2

```
cics002:Git_example jaimedavila$ vi sample_file.txt
cics002:Git_example jaimedavila$ cat sample_file.txt
this is juust a sample file
this line was added in the main branch
cics002:Git_example jaimedavila$ git commit -am "small change in one file on main branch"
[main a078b03] small change in one file on main branch
 1 file changed, 1 insertion(+)
cics002:Git_example jaimedavila$ git status
On branch main
nothing to commit, working tree clean
cics002:Git_example jaimedavila$ git diff main..v2
diff --git a/sample_file.txt b/sample_file.txt
index 9ac2fa9..ac7203b 100644
--- a/sample_file.txt
+++ b/sample_file.txt
@@ -1,2 +1,2 @@
 this is juust a sample file
-this line was added in the main branch
+this line was added in version 2
cics002:Git_example jaimedavila$
```

Version Control With Git

- to combine/merge two branches (while on branch main)
  - **`git merge v2`**
- In this case, it tries, but fails. It doesn't know which edits to keep.

```
cics002:Git_example jaimedavila$ git merge v2
Auto-merging sample_file.txt
CONFLICT (content): Merge conflict in sample_file.txt
Automatic merge failed; fix conflicts and then commit the result.
cics002:Git_example jaimedavila$
```

- And it actually "broke" our file

```
cics002:Git_example jaimedavila$ cat sample_file.txt
this is juust a sample file
<<<<<<< HEAD
this line was added in the main branch
=======
this line was added in version 2
>>>>>>> v2
cics002:Git_example jaimedavila$
```

Version Control With Git

- To get rid of that attempt to merge
  - **`git merge --abort`**

```
cics002:Git_example jaimedavila$ git merge --abort
cics002:Git_example jaimedavila$ cat sample_file.txt
this is juust a sample file
this line was added in the main branch
cics002:Git_example jaimedavila$
```

Version Control With Git

- Git is able, however, to combine simpler differences between branches.
- For example, if the common files are identical, and there's a file that exists in one branch but not the other, it will copy that file over.

```
cics002:Git_example jaimedavila$ git status
On branch v2
nothing to commit, working tree clean
cics002:Git_example jaimedavila$ vi another_file.txt
cics002:Git_example jaimedavila$ cat another_file.txt
 this is a second file on branch v2
cics002:Git_example jaimedavila$ git add another_file.txt
cics002:Git_example jaimedavila$ git commit -am "adding a second file on branch v2"
[v2 3ddda64] adding a second file on branch v2
 1 file changed, 1 insertion(+)
 create mode 100644 another_file.txt
cics002:Git_example jaimedavila$ ls
another_file.txt        sample_file.txt
cics002:Git_example jaimedavila$ git switch main
Switched to branch 'main'
cics002:Git_example jaimedavila$ ls
sample_file.txt
cics002:Git_example jaimedavila$ git merge v2
Merge made by the 'recursive' strategy.
 another_file.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 another_file.txt
cics002:Git_example jaimedavila$ ls
another_file.txt        sample_file.txt
cics002:Git_example jaimedavila$
```

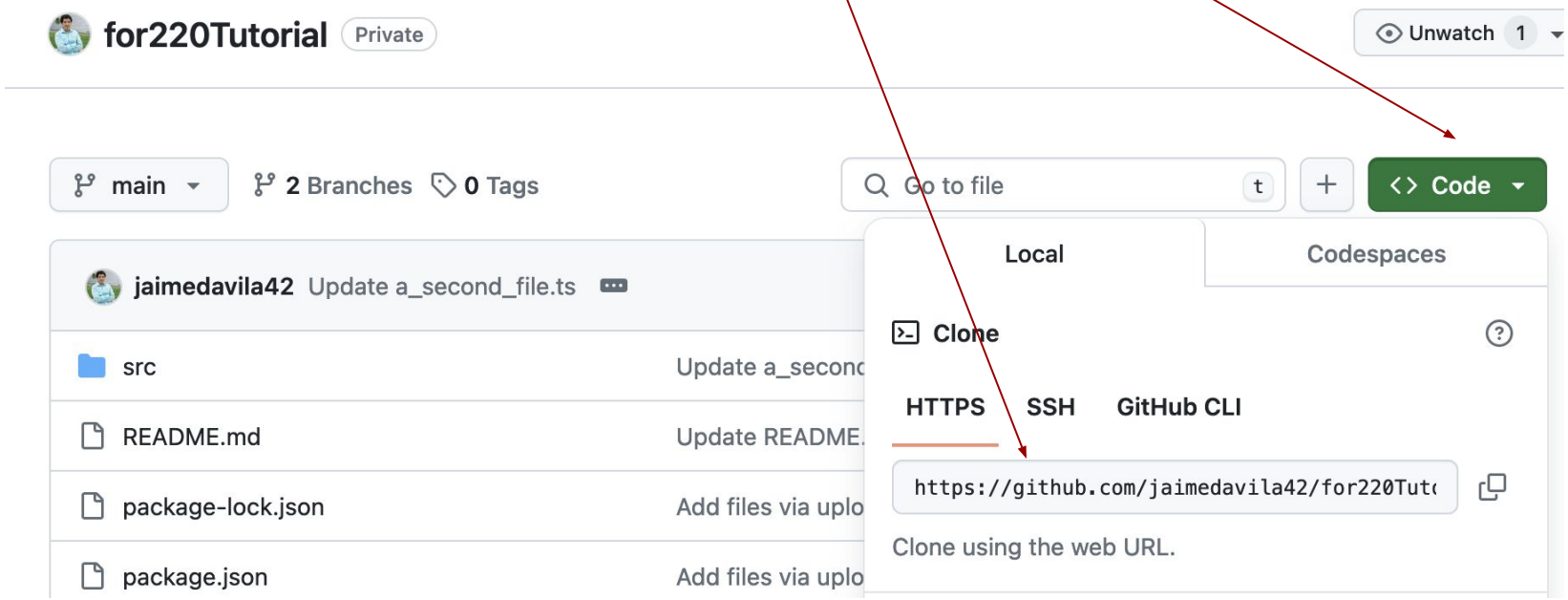
Connecting to GitHub

- The first time you connect to GitHub, you will need to authenticate.
  - It's not as easy as "this is my username, this is my password."
- How to create tokens: https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/creating-a-personal-access-token

Connecting to GitHub

Copy this (in your HW8 repo)

Click here



Connecting to GitHub

- Make a folder
- Change to that folder
- Clone the repository you were given access to for HW8.

```
cics002:Desktop jaimedavila$ mkdir From_git
cics002:Desktop jaimedavila$ cd From_git/
cics002:From_git jaimedavila$ git clone https://github.com/jaimedavila42/for220Tutorial.git
Cloning into 'for220Tutorial'...
remote: Enumerating objects: 26, done.
remote: Counting objects: 100% (26/26), done.
remote: Compressing objects: 100% (23/23), done.
remote: Total 26 (delta 8), reused 10 (delta 3), pack-reused 0
Receiving objects: 100% (26/26), 5.56 KiB | 2.78 MiB/s, done.
Resolving deltas: 100% (8/8), done.
cics002:From_git jaimedavila$ ls
for220Tutorial
cics002:From_git jaimedavila$
```

Files are now here